



People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research

Mohamed Seddik Ben Yahia University - Jijel

Faculty of Sciences and Technology
Department of Computer Science

School Management System

UML Analysis and Design Report

Specialty: Computer Science (Informatics)

Level: 3rd Year License

Academic Session: 2025 - 2026

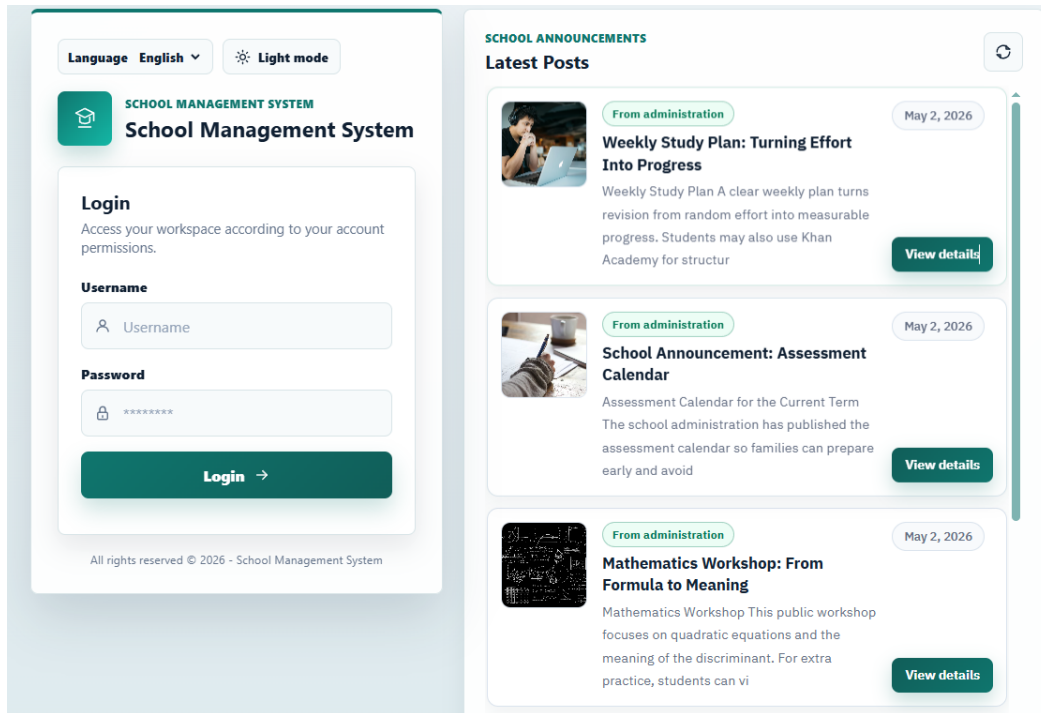
Prepared by

1. Bouremouz Mouhammed
2. Debieche Amine
3. Bessibes Mouin
4. Chaabani Sidahmed
5. Bouzekria Sohib
6. Meriche Chemseddine

Under the supervision of

Dr. El Hillali Kerkouche

Executive Summary



Login interface preview

The School Management System is a complete web application for organizing school administration, academic follow-up, communication, and finance. It is built around a frontend/backend architecture and six role-based dashboards: Admin, Receptionist, Accountant, Teacher, Student, and Parent.

Project goal	Digitize daily school workflows, reduce manual follow-up, and give each actor a secure workspace.
Architecture	Static HTML/CSS/JavaScript frontend connected to a Python FastAPI backend and relational database.
Frontend structure	Shared core modules manage API calls, authentication, routing, preferences, storage, and i18n; each role has its own page, services, controller, and UI renderer.
Backend structure	FastAPI routers and database managers cover users, parents, students, teachers, classes, enrollments, assignments, schedules, attendance, assessments, grades, resources, fees, payments, transactions, messages, notifications, posts, and programs.
Data design	The initial structure was studied from an Excel data reference from Al Abakera School, then transformed into relational entities and UML models.
Deployment	The project was prepared for hosted demonstration using frontend/backend port forwarding and cloud services for remote access.
Documentation	The UML report includes use case diagrams, class diagram, sequence diagrams, navigation diagram, technology notes, GitHub repository, and tooling notes.

Role Coverage

Admin	Manages users, parents, students, teachers, classes, programs, enrollments, attendance, schedules, assessments, grades, resources, finance, posts, notifications, and conversations.
Receptionist	Handles student and parent registration, student files, search, finance lookup, payments, posts, messages, and notifications.
Accountant	Follows student fees, payments, transactions, financial files, attendance reports, messages, notifications, and finance notices.

Teacher	Uses assignments, schedules, attendance sheets, assessments, grade entry, educational resources, posts, messages, and notifications.
Student	Consults schedule, assessments, grades, attendance, resources, fees, posts, messages, and notifications.
Parent	Follows linked children, grades, attendance, fees, posts, messages, and notifications.

In short, this summary gives the reader the project scope, technical structure, main features, data-modeling approach, deployment context, and UML documentation strategy before reading the detailed diagrams and sections.

Table of Contents

Introduction	5
Project Context and Benefits	5
Functional Models: Use Case Diagrams	5
Admin Use Case Diagram	7
Receptionist Use Case Diagram	8
Accountant Use Case Diagram	9
Teacher Use Case Diagram	10
Student Use Case Diagram	11
Parent Use Case Diagram	12
Structural Model: Class Diagram	13
School Management System Class Diagram	14
Dynamic Models: Sequence Diagrams	15
Admin Sequence Diagram	16
Receptionist Sequence Diagram	17
Accountant Sequence Diagram	18
Teacher Sequence Diagram	19
Student Sequence Diagram	20
Parent Sequence Diagram	21
User Interface: Navigation Diagram	22
Role-Based Navigation Diagram	23
Technology Stack, Deployment, and Repository	24
Conclusion	24

Introduction

This report presents the analysis and design of a web-based School Management System. The application is organized around six role-based dashboards: Admin, Receptionist, Accountant, Teacher, Student, and Parent.

The documentation includes a concise project context, use case diagrams, a class diagram, sequence diagrams, a navigation diagram, technology/deployment notes, and a conclusion.

Project Context and Benefits

Opening Statement

The project is a role-based web application for school management. It connects administration, reception, accounting, teachers, students, and parents through one frontend/backend platform.

System Benefits

Benefit	Contribution to the School Management System
Administrative efficiency	Centralizes records, enrollments, schedules, and follow-up.
Pedagogical follow-up	Tracks assessments, grades, attendance, and resources.
Communication	Uses messages, notifications, and administration posts.
Financial clarity	Supports fees, payments, transactions, and reminders.
Role-based organization	Separates dashboards by responsibility and user role.

Initial Data Modeling with AI Abakera Excel Data

At the beginning, an Excel data reference from AI Abakera School was used to identify the main entities: users, students, parents, teachers, classes, enrollments, fees, payments, attendance, schedules, assessments, grades, and resources.

Hosting, Port Forwarding, and Cloud Services

Port forwarding was configured for both frontend and backend services to support remote testing. Cloud services were used to make the hosted demonstration accessible outside the local network.

Functional Models: Use Case Diagrams

The use case diagrams define the visible services of the School Management System for each role. They show the functional boundary of every dashboard and clarify which operations belong to administration, reception, accounting, teaching, learner self-service, and parent follow-up.

Use Case Scope

Admin: complete system administration across people, academic, finance, posts, and notifications.

Receptionist: front-desk workflows for students, parents, records, finance lookup, posts, and communication.

Accountant: fees, payments, transactions, student financial files, notices, and attendance reports.

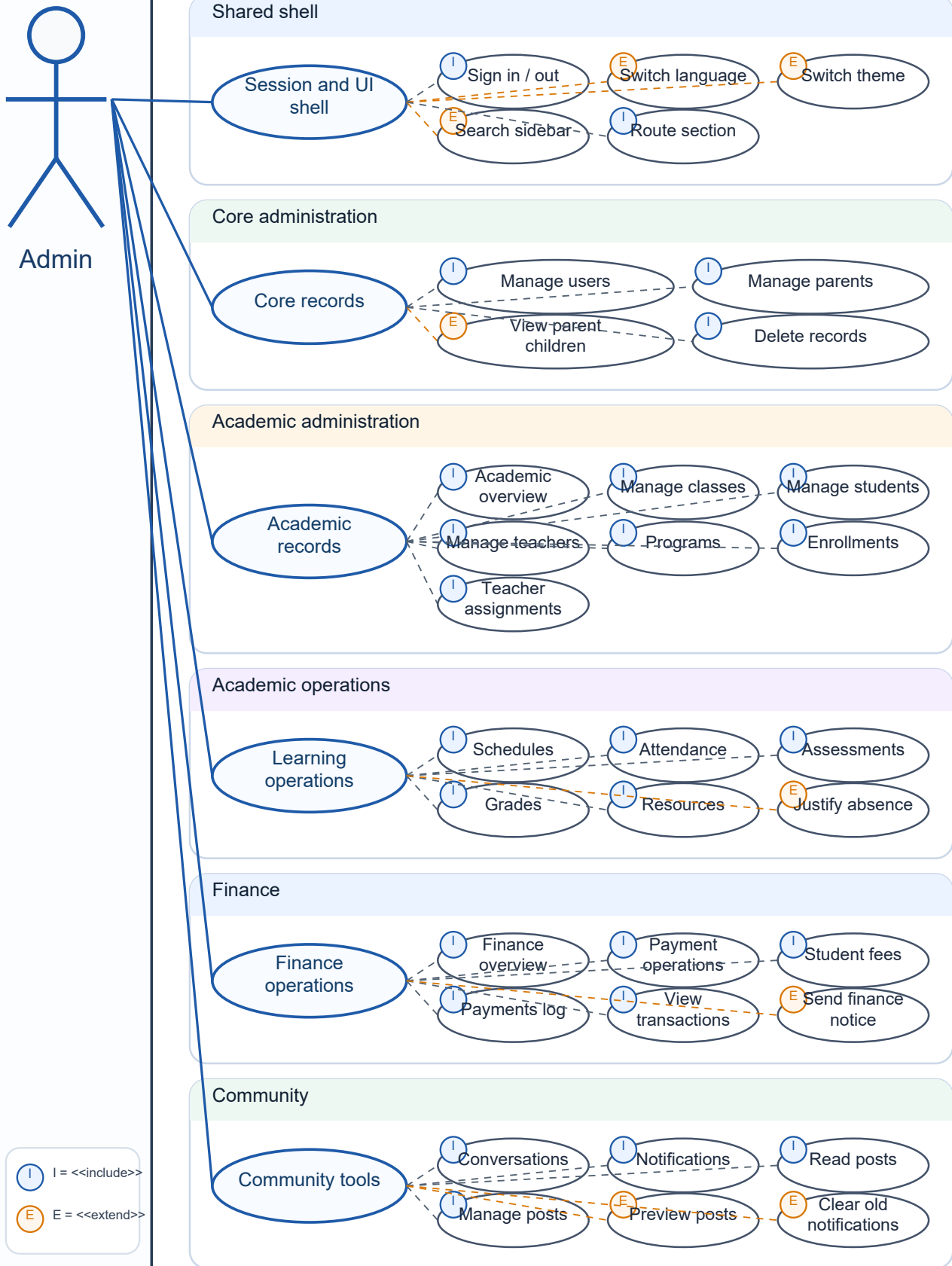
Teacher: assignments, schedules, attendance, assessments, grades, resources, messages, and notifications.

Student: read-only academic self-service: schedule, assessments, grades, attendance, resources, posts, fees, messages, and notifications.

Parent: child monitoring: grades, attendance, fees, posts, messages, and notifications.

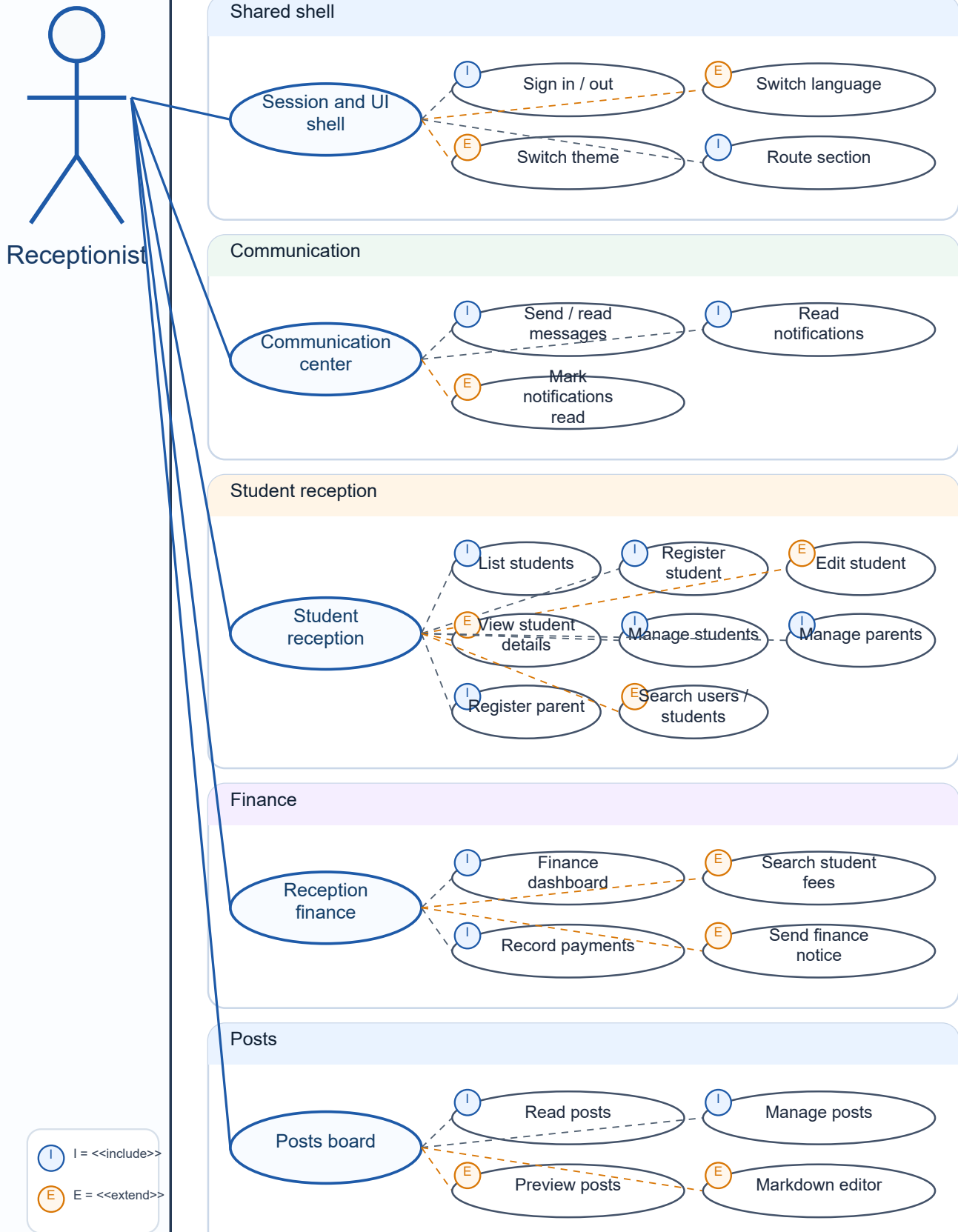
Use Case Diagram - Admin

School Management System boundary



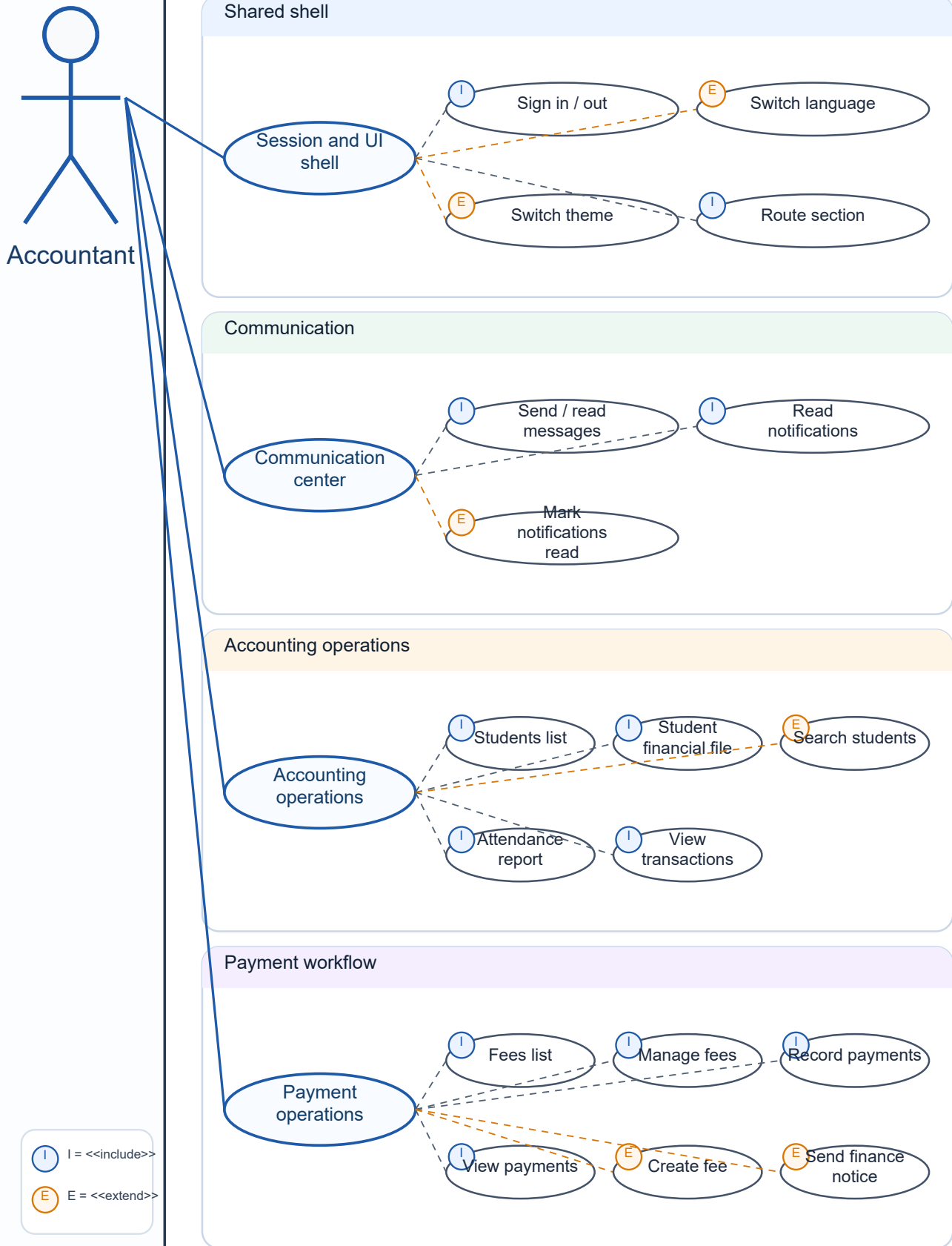
Use Case Diagram - Receptionist

School Management System boundary



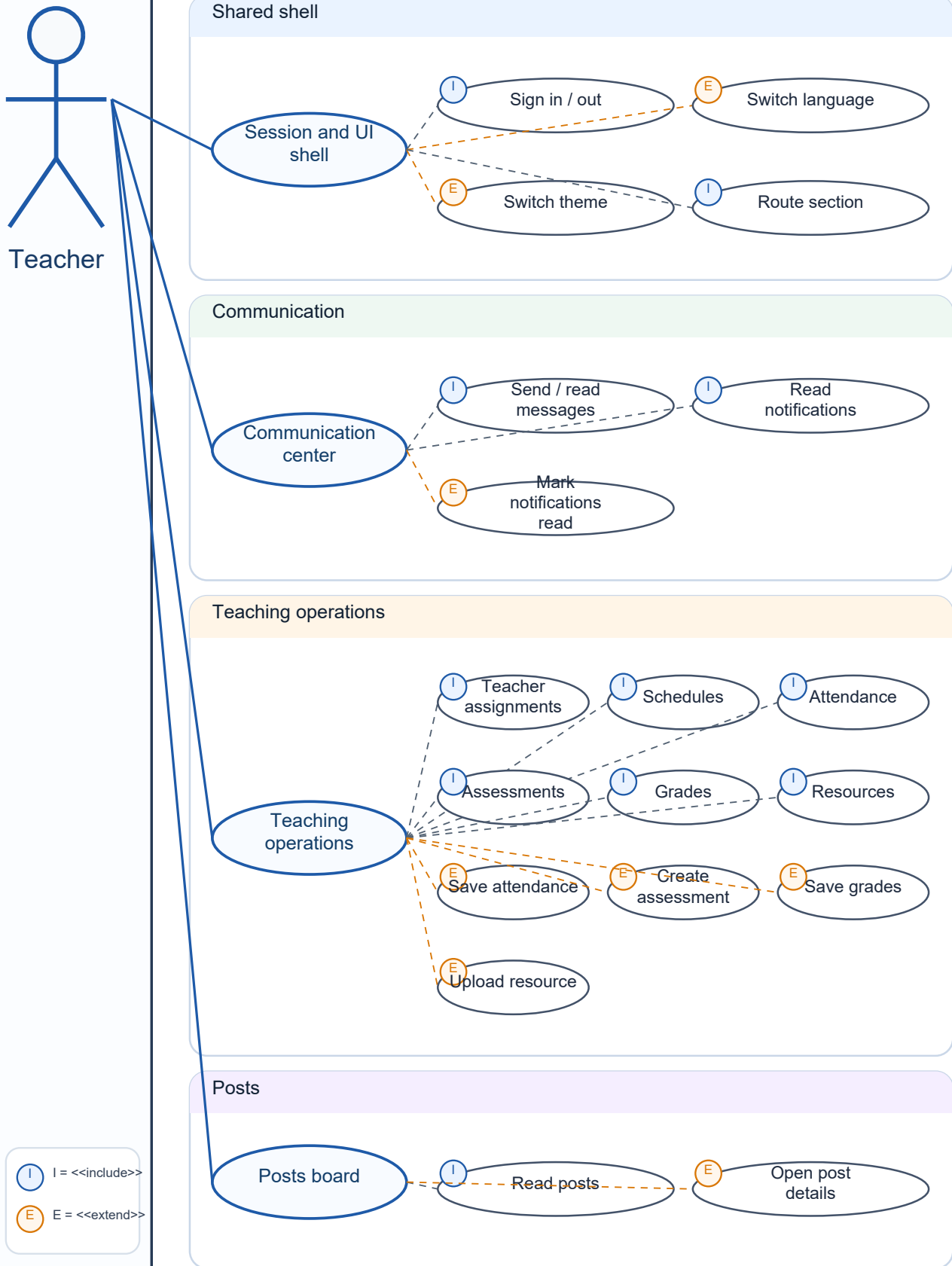
Use Case Diagram - Accountant

School Management System boundary



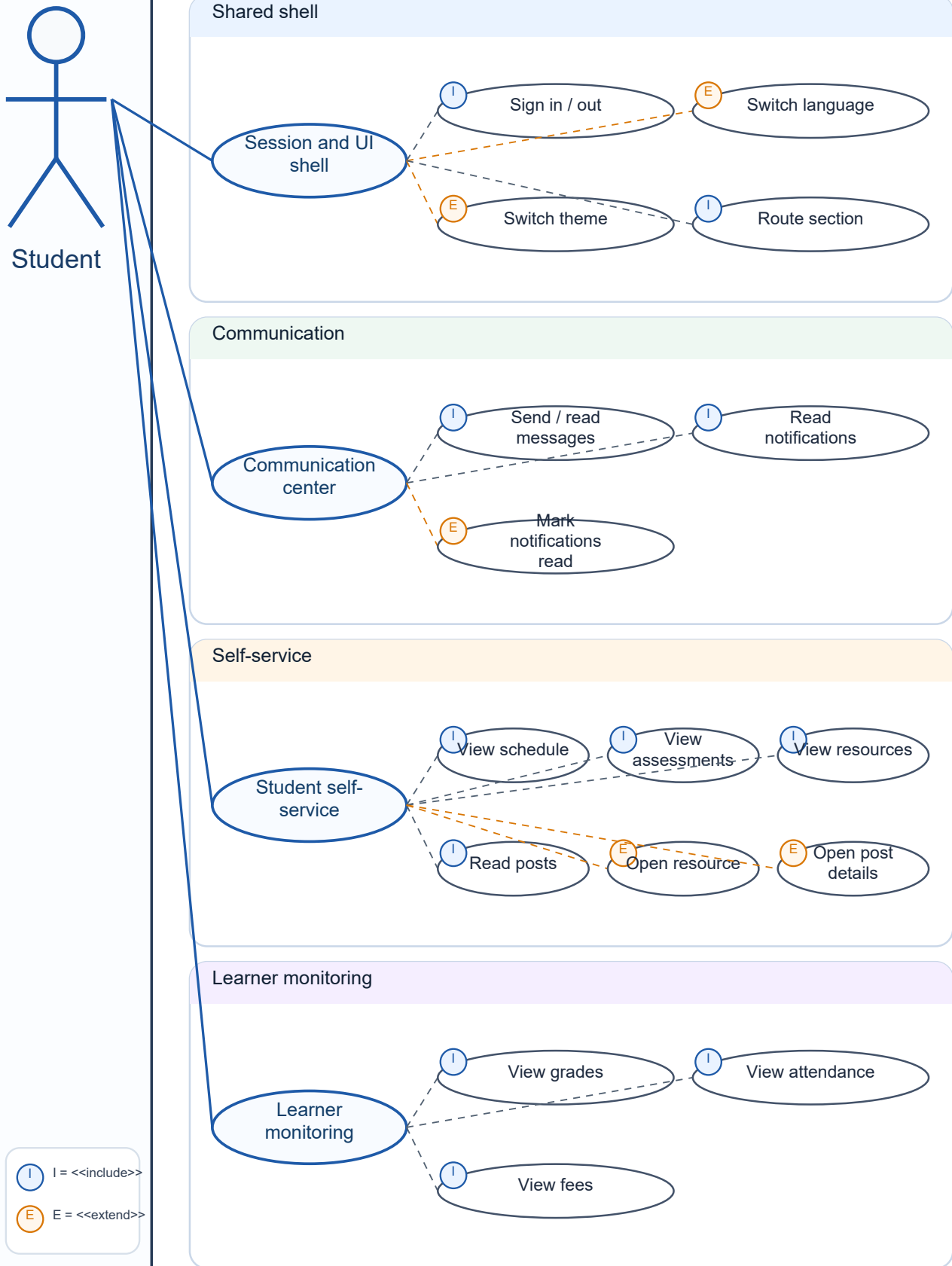
Use Case Diagram - Teacher

School Management System boundary



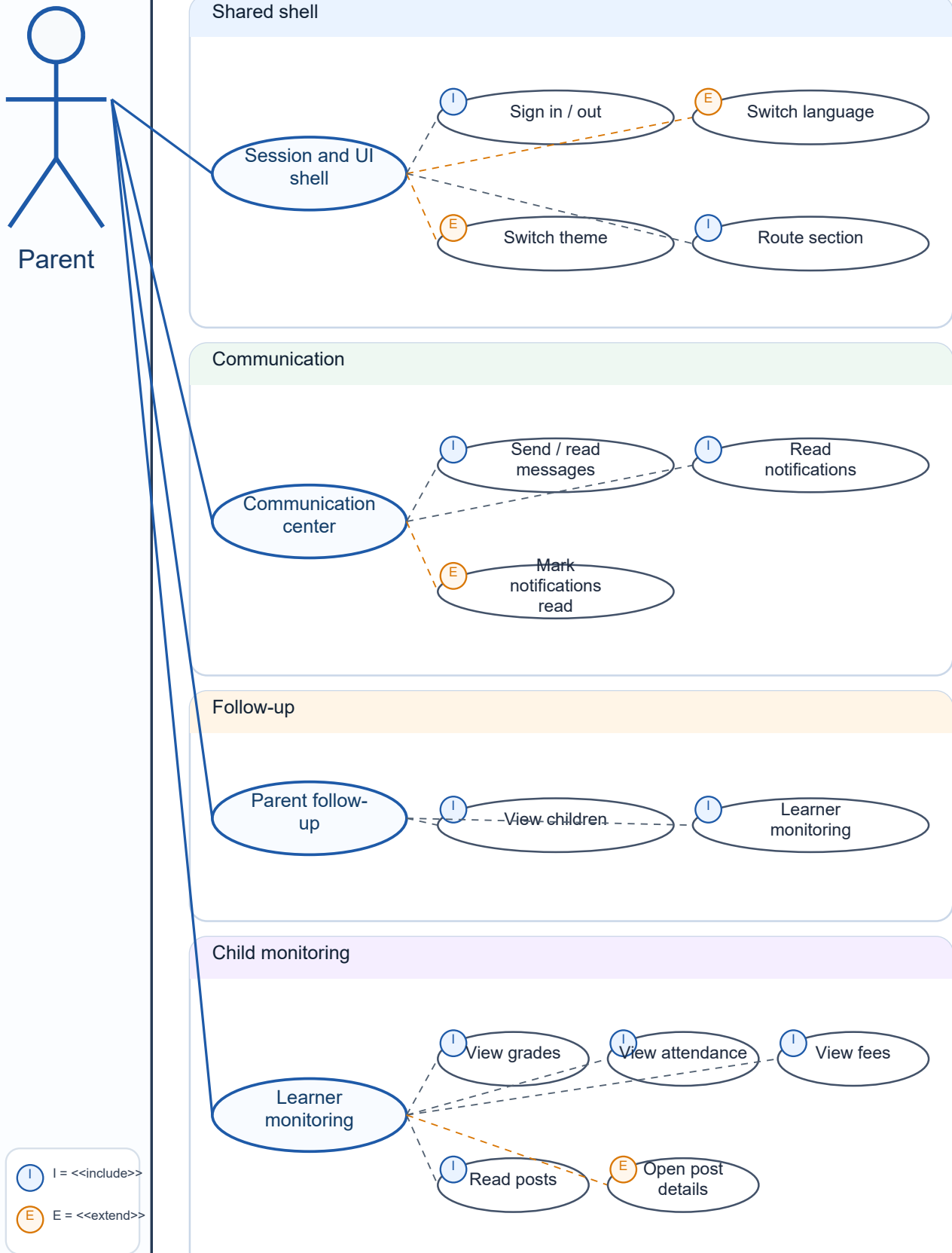
Use Case Diagram - Student

School Management System boundary



Use Case Diagram - Parent

School Management System boundary



Structural Model: Class Diagram

The class diagram presents the backend structure of the School Management System. It groups the database managers and domain entities used to support identity, academic management, learning operations, finance, communication, and publication features.

Structural Responsibilities

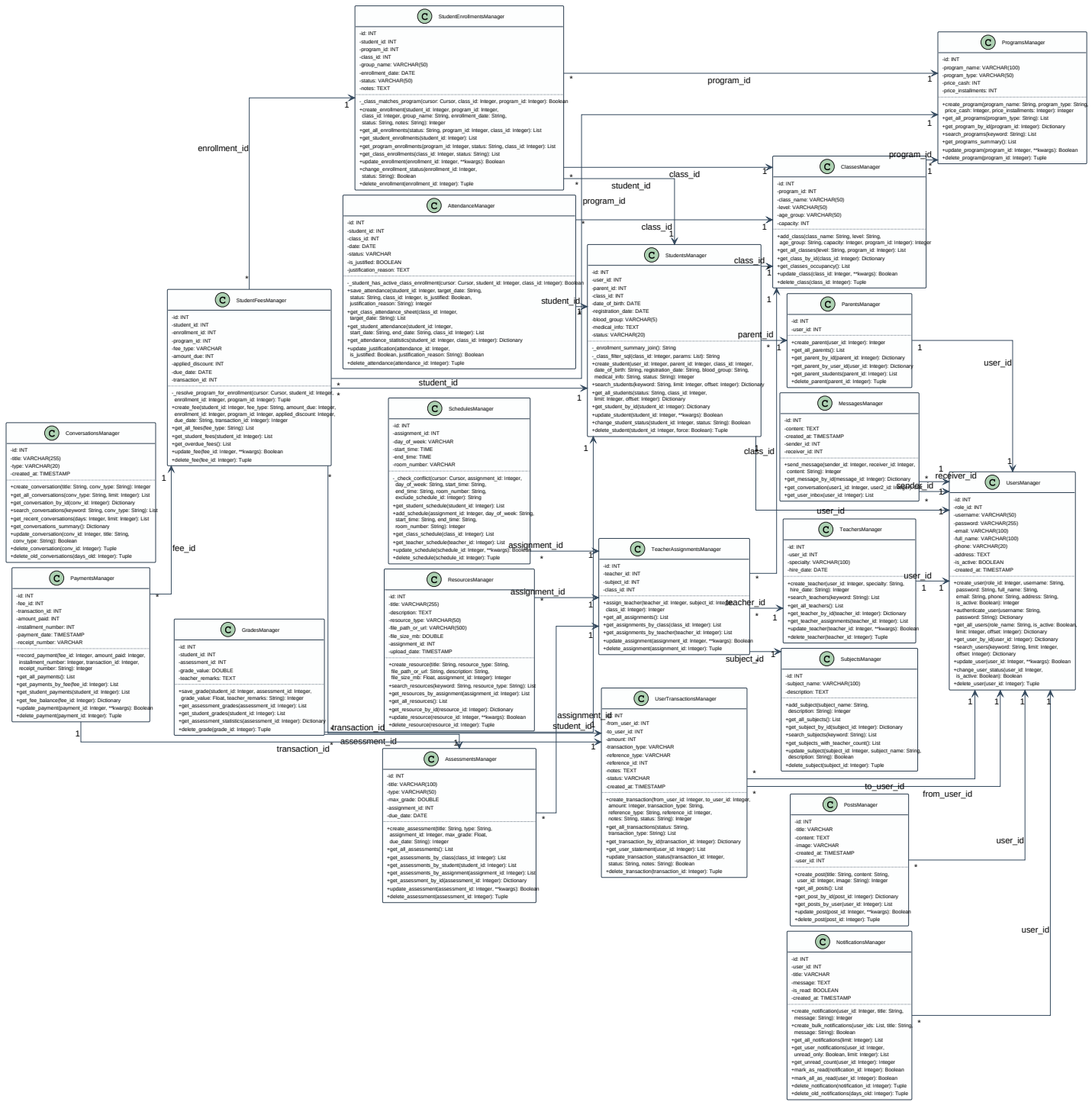
Identity and people: users, parents, students, teachers, authentication, and role ownership.

Academic structure: programs, classes, enrollments, subjects, and teacher assignments.

Learning operations: schedules, attendance, assessments, grades, and educational resources.

Finance: student fees, payments, and user transactions.

Communication: conversations, messages, notifications, and posts.



Dynamic Models: Sequence Diagrams

The sequence diagrams describe how each role interacts with the application at runtime. The common pattern is: user action in the frontend, role controller handling, service/API request, FastAPI router processing, database operation, then UI rendering of the result.

Sequence Flow Summary

Admin: authentication, route selection, data loading, and full create/update/delete administration.

Receptionist: dashboard bootstrap, student and parent registration, student records, payments, posts, messaging, and notifications.

Accountant: finance workspace loading, student financial files, fee creation, payment recording, notices, attendance reports, messaging, and notification updates.

Teacher: teacher profile resolution, assignments, schedules, attendance, assessments, grades, resources, messaging, and notifications.

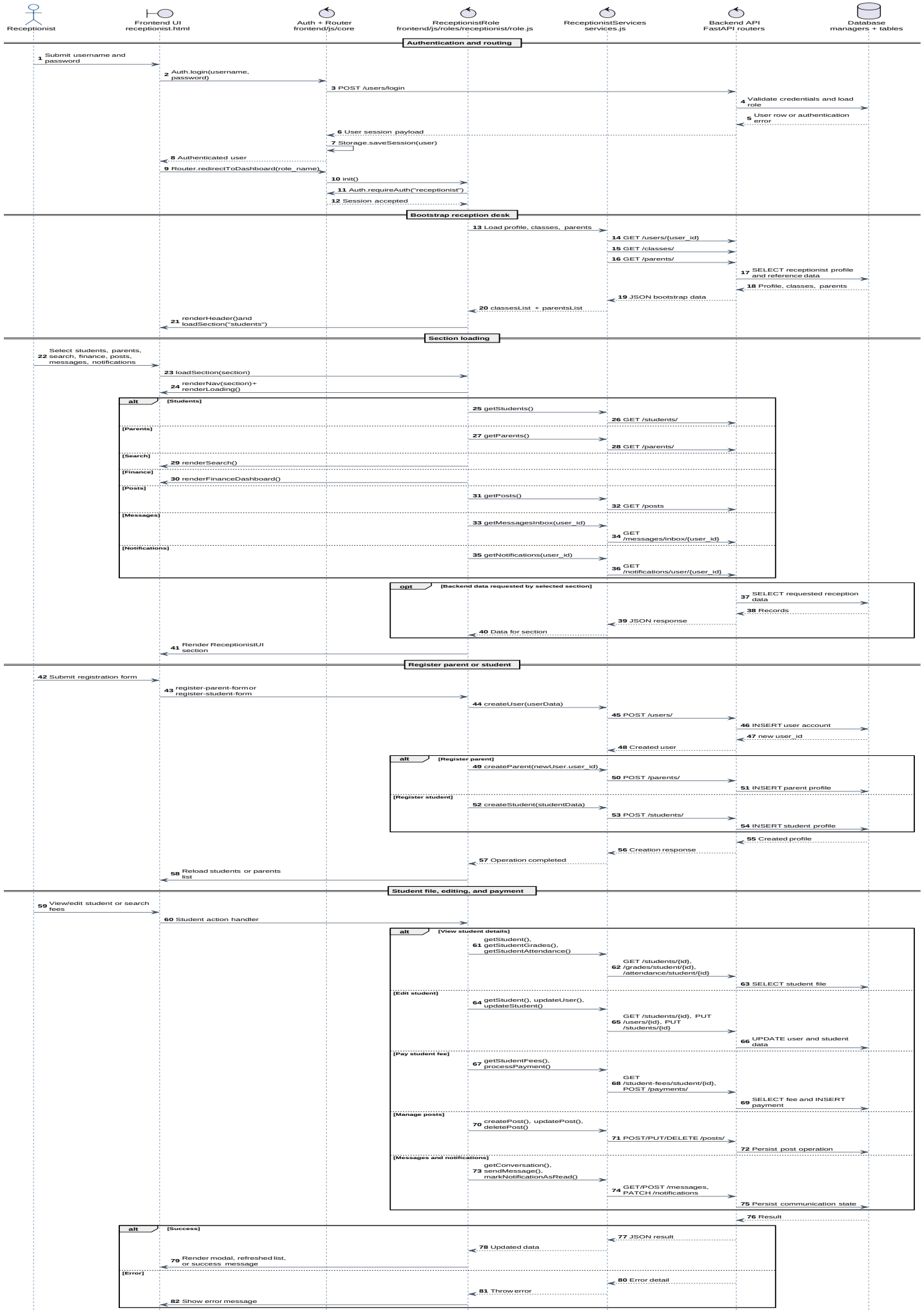
Student: linked profile resolution, academic self-service views, messages, and notifications.

Parent: parent profile resolution, linked children aggregation, child monitoring, posts, messages, and notifications.

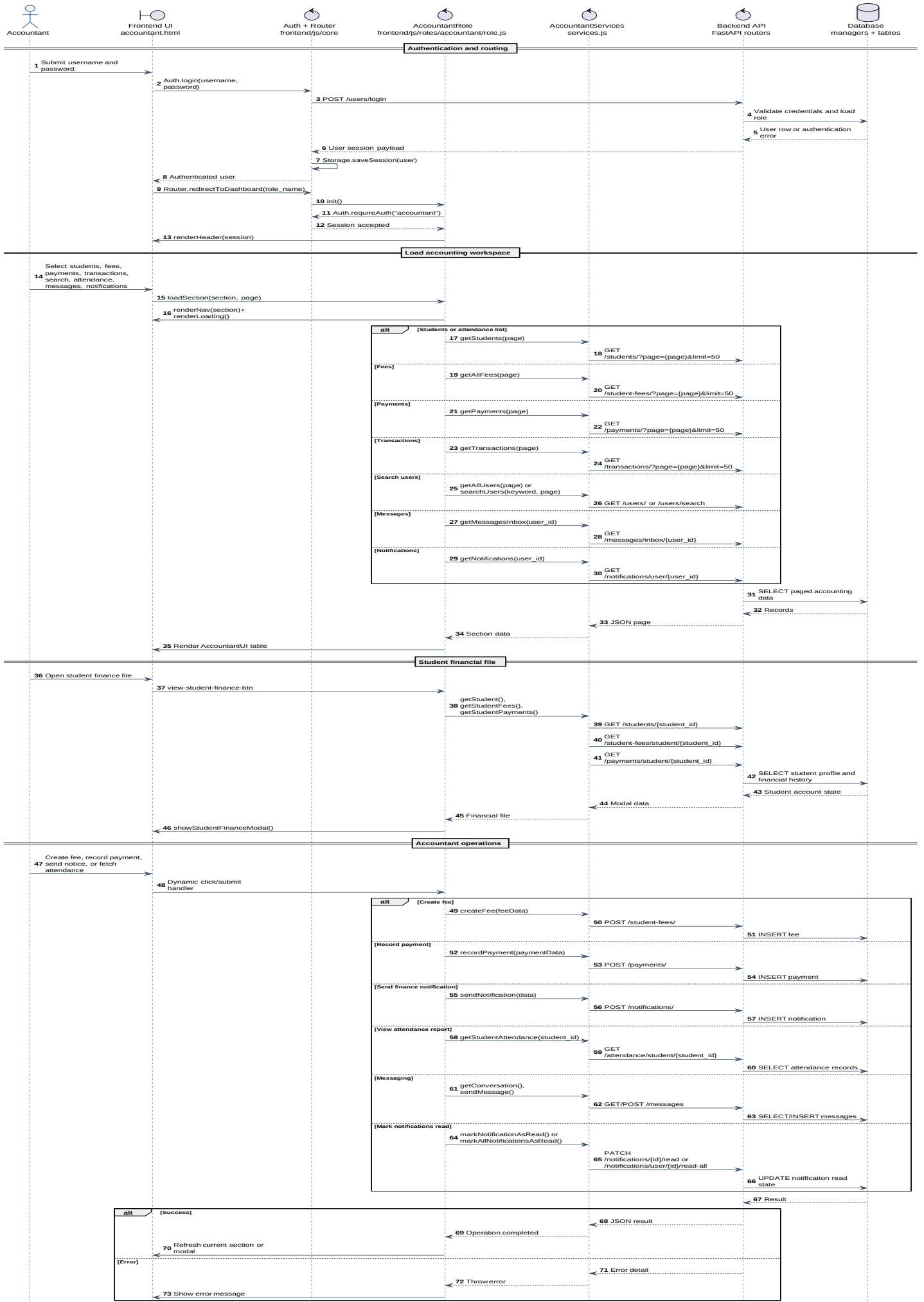
Sequence Diagram - Admin



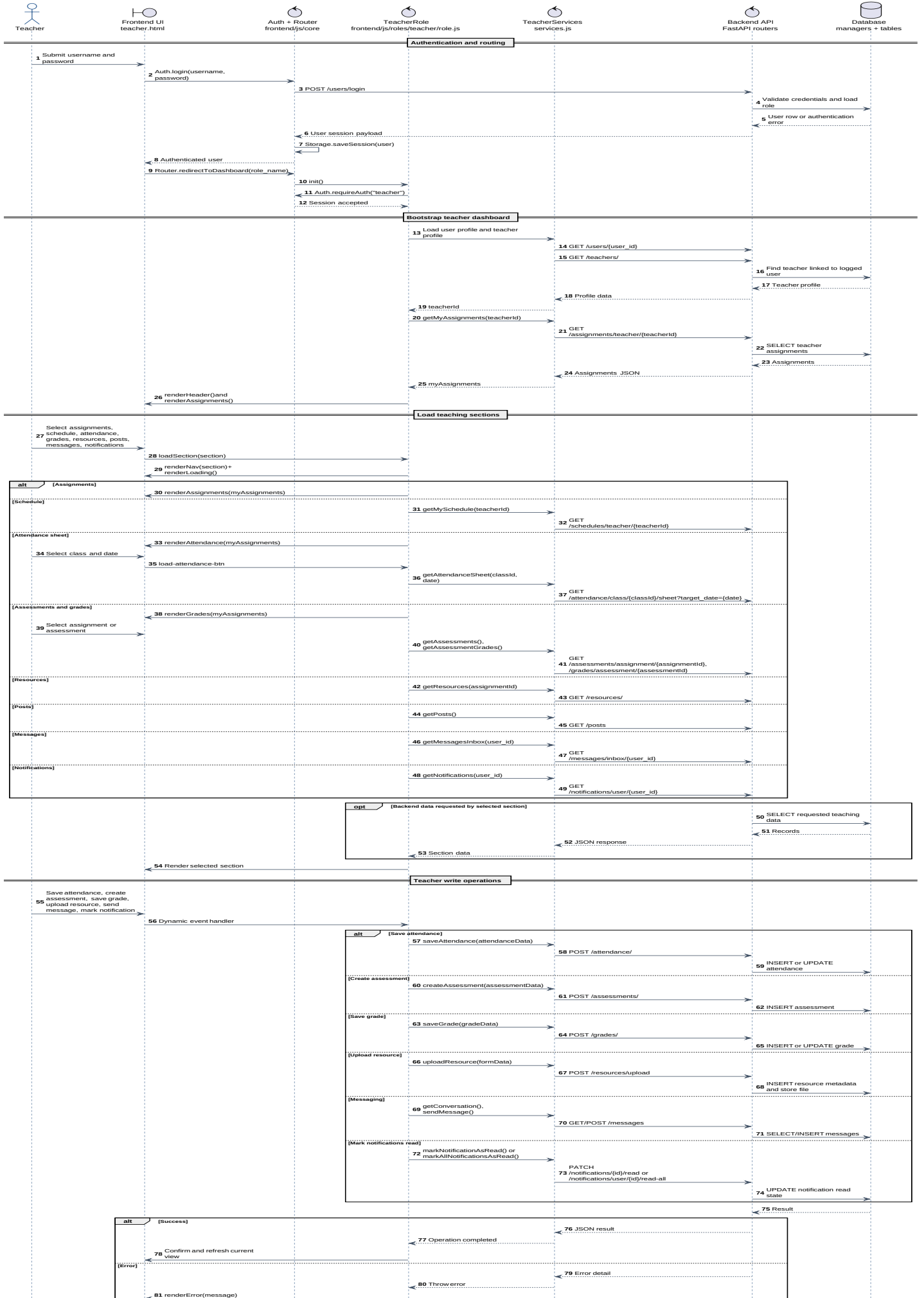
Sequence Diagram - Receptionist



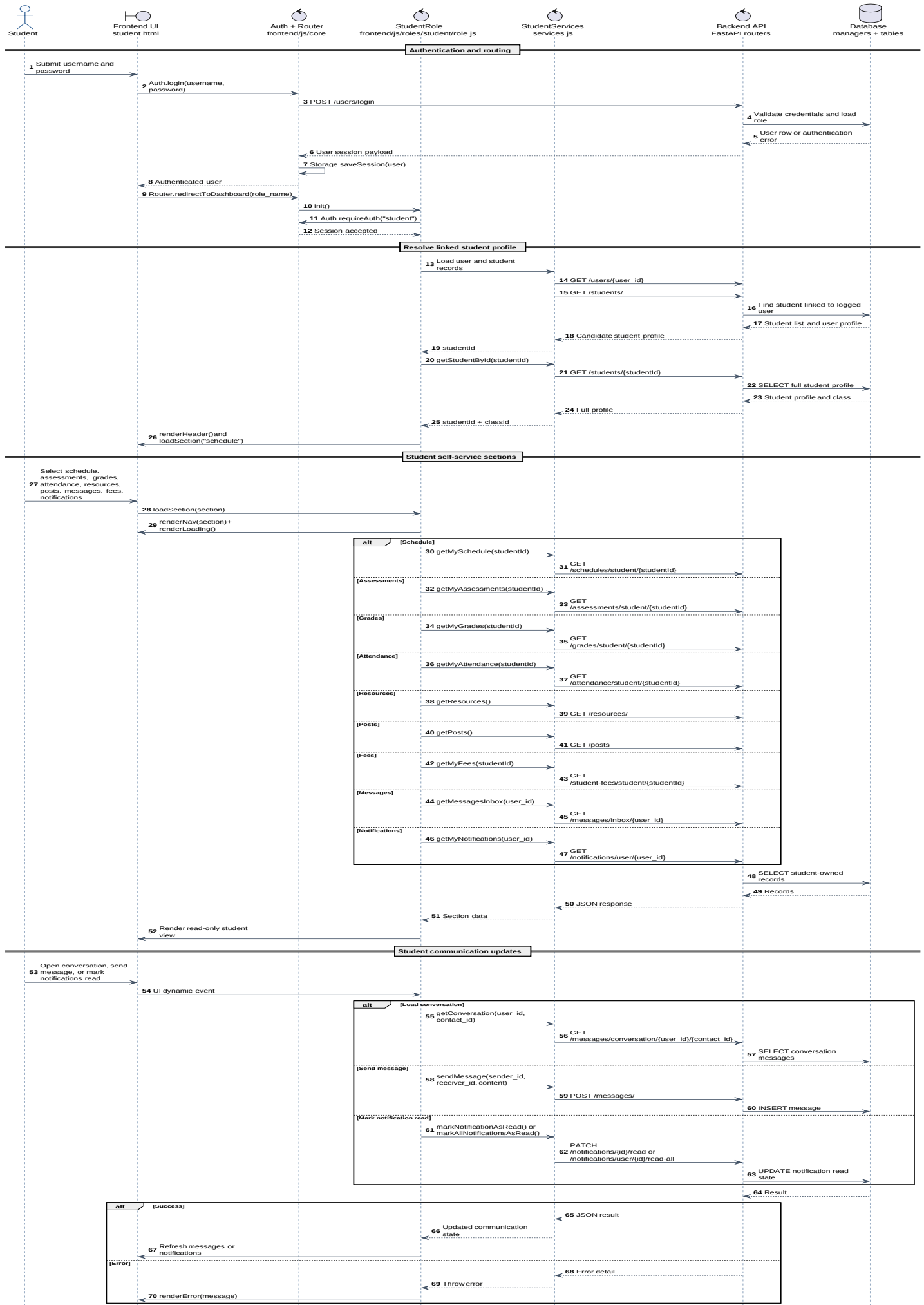
Sequence Diagram - Accountant



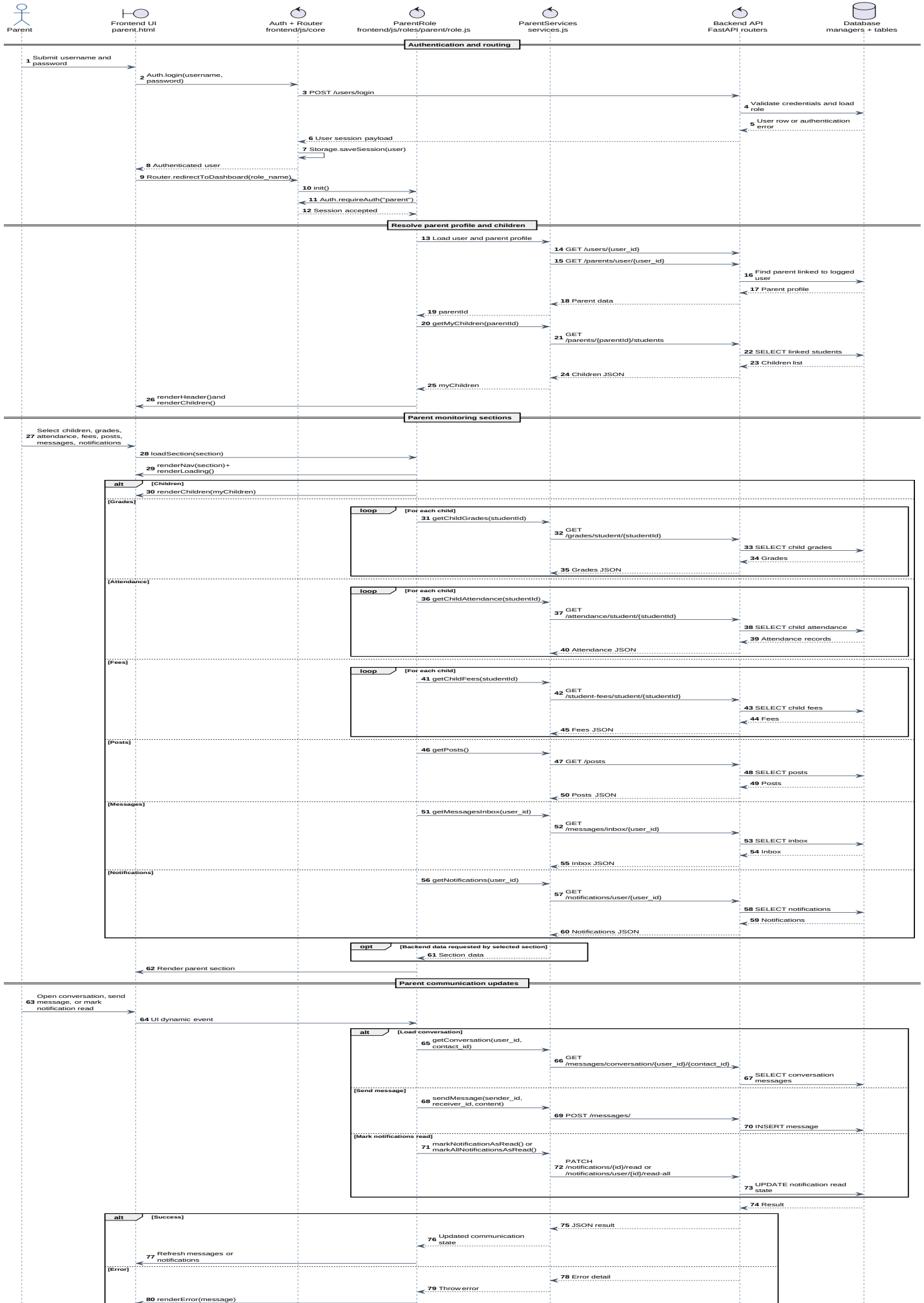
Sequence Diagram - Teacher



Sequence Diagram - Student



Sequence Diagram - Parent



User Interface: Navigation Diagram

The navigation diagram documents how users move from the entry point and authentication page into their role-specific dashboards. It also shows the shared frontend controls and the major section families exposed by each role.

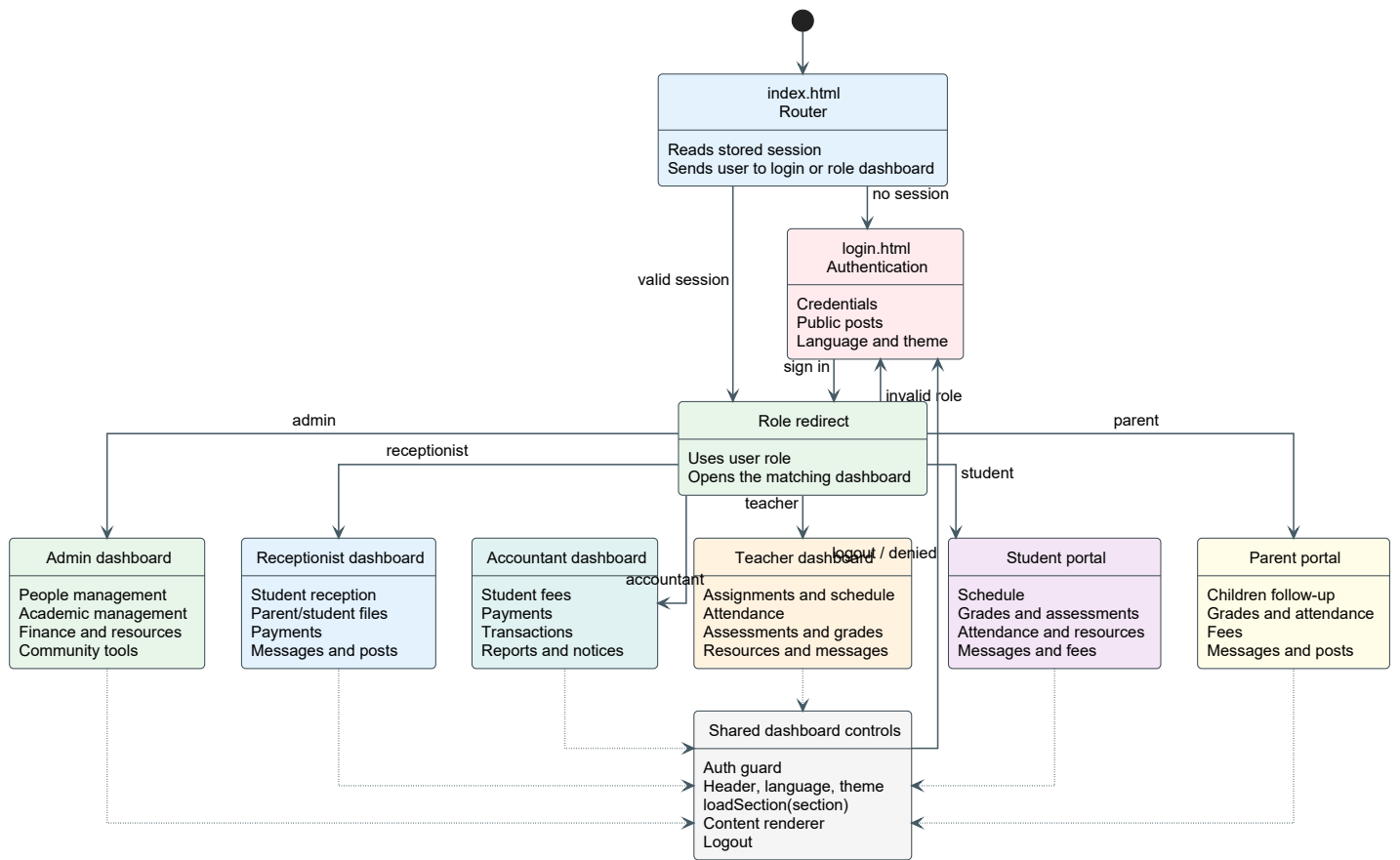
Navigation Flow

Entry: users start from the index/login flow, where authentication determines the target dashboard.

Dashboards: Admin, Receptionist, Accountant, Teacher, Student, and Parent each have a dedicated page and role controller.

Sections: navigation buttons call the active role controller, which loads the selected section and renders the corresponding UI.

Shared controls: authentication checks, language switching, theme preferences, logout, messaging, and notifications are reused across dashboards.



Technology Stack, Deployment, and Repository

This section lists the main technologies, deployment notes, and repository reference.

Frontend Layer

The frontend is a static web application built with **HTML5**, **CSS3**, and **vanilla JavaScript**. It uses shared core modules for routing, authentication, local storage, API calls, preferences, and internationalization. Each role has its own page, controller, service layer, and UI renderer.

Backend API Layer

The backend is built with **Python** and **FastAPI**. The API is organized into routers for users, parents, students, teachers, classes, enrollments, schedules, attendance, assessments, grades, resources, fees, payments, transactions, notifications, messages, posts, and programs. **Uvicorn** is used as the ASGI server and **Pydantic** supports request/response validation.

Database and Persistence

The persistence layer uses a relational database accessed through **mysql-connector-python** and **SQLAlchemy** utilities. Database managers separate identity, academic, learning, finance, and communication responsibilities.

Documentation and UML Report

UML diagrams are exported as **SVG** assets and embedded into the PDF report with **ReportLab** and **svglib**. The report covers use case diagrams, the class diagram, sequence diagrams, and the navigation diagram.

Hosting and Cloud Access

Port forwarding was configured for the frontend and backend to enable remote testing. Cloud services supported the hosted demonstration.

Artificial Intelligence Assistance

AI-assisted tools supported analysis, code review, UML/report refinement, and documentation.

Project Repository

The project source code is available on GitHub:
<https://github.com/MohammedBoure/SMS-FE-BE>

Conclusion

Project Summary

The UML documentation gives a structured view of the system: role responsibilities, backend structure, frontend-to-backend interactions, and navigation flow.

Design Value

The diagrams make the project easier to review and extend. The data structure was first studied from the AI Abakera Excel reference, then modeled into UML, backend APIs, frontend dashboards, and a hosted web setup.

Repository and Tooling Note

The project repository is available at <https://github.com/MohammedBoure/SMS-FE-BE>. AI-assisted tools supported analysis, documentation refinement, and UML/report quality improvements.